

GPSTracker

Paz para Você, Proteção para Sua Carga. Tecnologia a Seu Favor.

Generated by Doxygen 1.9.8

1 Namespace Index	1
1.1 Namespace List	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Namespace Documentation	7
4.1 GPSTracker Namespace Reference	7
4.1.1 Variable Documentation	7
4.1.1.1 gps_monitor	7
5 Class Documentation	9
5.1 GPSTracker::GPSData Class Reference	9
5.1.1 Detailed Description	10
5.1.2 Constructor & Destructor Documentation	11
5.1.2.1 GPSData()	11
5.1.3 Member Function Documentation	11
5.1.3.1 converter_lat_lon()	11
5.1.3.2 parsing()	11
5.1.3.3 to_csv()	12
5.1.4 Member Data Documentation	12
5.1.4.1 data	12
5.1.4.2 pattern	12
5.2 GPSTracker::GPSMonitor Class Reference	13
5.2.1 Detailed Description	14
5.2.2 Constructor & Destructor Documentation	14
5.2.2.1 __init__()	14
5.2.3 Member Function Documentation	14
5.2.3.1 conversion_utc_to_brazil()	14
5.2.3.2 get_data_from_client()	15
5.2.3.3 mainloop()	15
5.2.3.4 save_historic()	15
5.2.3.5 set_background()	16
5.2.4 Member Data Documentation	16
5.2.4.1 altitude	16
5.2.4.2 img_data	16
5.2.4.3 placeholder	16
5.2.4.4 sock	16
5.2.4.5 time	16
5.2.4.6 trajet	16
5.2.4.7 zona_brasileira	17

5.3 GPSSim Class Reference	17
5.3.1 Detailed Description	18
5.3.2 Constructor & Destructor Documentation	19
5.3.2.1 GPSSim()	19
5.3.2.2 ~GPSSim()	19
5.3.3 Member Function Documentation	19
5.3.3.1 get_path_pseudo_term()	19
5.3.3.2 init()	20
5.3.3.3 loop()	20
5.3.3.4 stop()	20
5.3.3.5 update()	20
5.3.4 Member Data Documentation	20
5.3.4.1 alt	20
5.3.4.2 caminho_do_pseudo_terminal	21
5.3.4.3 fd_filho	21
5.3.4.4 fd_pai	21
5.3.4.5 is_exec	21
5.3.4.6 lat	21
5.3.4.7 lon	21
5.3.4.8 to_move	21
5.3.4.9 worker	22
5.4 GPSTracker Class Reference	22
5.4.1 Detailed Description	24
5.4.2 Constructor & Destructor Documentation	25
5.4.2.1 GPSTracker()	25
5.4.2.2 ~GPSTracker()	25
5.4.3 Member Function Documentation	26
5.4.3.1 init()	26
5.4.3.2 loop()	26
5.4.3.3 open_serial()	26
5.4.3.4 read_serial()	27
5.4.3.5 send()	27
5.4.3.6 split()	27
5.4.3.7 stop()	28
5.4.4 Member Data Documentation	28
5.4.4.1 addr_dest	28
5.4.4.2 fd_serial	28
5.4.4.3 ip_destino	28
5.4.4.4 is_exec	28
5.4.4.5 last_data_given	28
5.4.4.6 porta_destino	29
5.4.4.7 porta_serial	29

5.4.4.8 sockfd	29
5.4.4.9 worker	29
5.5 GPSSim::NMEAGenerator Class Reference	29
5.5.1 Detailed Description	30
5.5.2 Member Function Documentation	31
5.5.2.1 build_nmea_string()	31
5.5.2.2 degrees_to_NMEA()	31
5.5.2.3 format_integer()	31
5.5.2.4 generate_gga()	32
5.5.2.5 generate_rmc()	32
5.5.2.6 get_utc_time()	33
6 File Documentation	35
6.1 src/debug.cpp File Reference	35
6.1.1 Detailed Description	35
6.1.2 Function Documentation	35
6.1.2.1 main()	35
6.2 debug.cpp	36
6.3 src/GPSMonitor.py File Reference	36
6.3.1 Detailed Description	37
6.4 GPSMonitor.py	37
6.5 src/GPSSim.hpp File Reference	39
6.5.1 Detailed Description	40
6.6 GPSSim.hpp	41
6.7 src/GPSTracker.hpp File Reference	44
6.7.1 Detailed Description	45
6.8 GPSTracker.hpp	46
6.9 src/main.cpp File Reference	50
6.9.1 Detailed Description	51
6.9.2 Function Documentation	51
6.9.2.1 main()	51
6.10 main.cpp	51
Index	53

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

GPSPMonitor	7
---------------------------------------	---

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

GPSTracker::GPSData	
Classe responsável por representar os dados do GPS	9
GPSSim.GPSMonitor	
Monitor que apresentará os dados do sensor	13
GPSSim	
Versão Simulada do Sensor GPS, gerando frases no padrão NMEA	17
GPSTracker	
Classe responsável por obter o tracking da carga	22
GPSSim::NMEAGenerator	
Classe responsável por agrupar as funções geradoras de sentenças NMEA	29

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

src/ debug.cpp	
Responsável por prover ferramentas de debug	35
src/ GPSTMonitor.py	
Implementação da interface de visualização de dados	36
src/ GPSSim.hpp	
Implementação da Classe Simuladora do GPS6MV2	39
src/ GPSTracker.hpp	
Implementação da solução embarcada	44
src/ main.cpp	
Responsável por executar a aplicação	50

Chapter 4

Namespace Documentation

4.1 GPSTest Namespace Reference

Classes

- class [GPSTest](#)
Monitor que apresentará os dados do sensor.

Variables

- str [gps_monitor](#) = [GPSTest](#)(img="src/antena.jpg", host="127.0.0.1", port=5000) if "self" not in st.session_state else st.session_state["self"]

4.1.1 Variable Documentation

4.1.1.1 gps_monitor

```
st.GPSTest.gps_monitor = GPSTest(img="src/antena.jpg", host="127.0.0.1", port=5000) if  
"self" not in st.session_state else st.session_state["self"]
```

Definition at line [205](#) of file [GPSTest.py](#).

Chapter 5

Class Documentation

5.1 GPSTracker::GPSData Class Reference

Classe responsável por representar os dados do GPS.

```
#include <GPSTracker.hpp>
```

Collaboration diagram for GPSTracker::GPSData:

GPSTracker::GPSData
- std::string pattern - std::vector< std::string > data
+ GPSData() + bool parsing(int code_pattern, const std::vector< std::string > &dataSplitted) + std::string to_csv() const + static std::string converter_lat_lon(const std::string &string_numerica, const std::string &string_hemisf)

Public Member Functions

- [GPSData](#) ()
Construtor Default.
- bool [parsing](#) (int code_pattern, const std::vector< std::string > &dataSplitted)
Setará os dados baseado no padrão de mensagem recebido.
- std::string [to_csv](#) () const
Retorna os dados armazenados em formato CSV.

Static Public Member Functions

- static std::string [converter_lat_lon](#) (const std::string &string_numerica, const std::string &string_hemisf)
Função estática auxiliar para converter coordenadas NMEA (latitude/longitude) para graus decimais.

Private Attributes

- std::string [pattern](#)
- std::vector< std::string > [data](#)
Organizaremos os dados neste vetor.

5.1.1 Detailed Description

Classe responsável por representar os dados do GPS.

Utilizar struct mostrou-se básico demais para atender as necessidades de representação dos dados do GPS. Com isso, a evolução para Class tornou-se inevitável.

O sensor inicia a comunicação enviando mensagens do tipo \$GPTXT. Essas mensagens são mensagens de texto informativas, geralmente vazias. Esse é um comportamento normal nos primeiros segundos após energizar o módulo.

Após adquirir sinais de satélites e iniciar a navegação, o módulo passa a enviar sentenças NMEA padrão, que trazem informações úteis:

- \$GPRMC (Recommended Minimum Navigation Information): Fornece dados essenciais de navegação, como horário UTC, status, latitude, longitude, velocidade, curso e data.
- \$GPVTG (Course over Ground and Ground Speed): Informa direção do movimento (rumo) e velocidade sobre o solo.
- \$GPGGA (Global Positioning System Fix Data): Dados de fixação do GPS, incluindo número de satélites usados, qualidade do sinal, altitude e posição.
- \$GPGSA (GNSS DOP and Active Satellites): Status dos satélites em uso e precisão (DOP - Dilution of Precision).
- \$GPGSV (GNSS Satellites in View): Informações sobre os satélites visíveis, como elevação, azimute e intensidade de sinal.
- \$GPGLL (Geographic Position - Latitude/Longitude): Posição geográfica em latitude e longitude, com horário associado.

Sendo assim, o sensor sai de um modo de inicialização para operacionalidade completa.

Como nosso propósito é apenas localização, nos interessa apenas o padrão GGA, o qual oferece dados profundos de localização.

Definition at line 99 of file [GPSTracker.hpp](#).

5.1.2 Constructor & Destructor Documentation

5.1.2.1 GPSData()

```
GPSTracker::GPSData::GPSData ( ) [inline]
```

Construtor Default.

Reserva no vetor de dados 4 strings, respectivamente para horário UTC, latitude, longitude e altitude.

Definition at line 114 of file [GPSTracker.hpp](#).

5.1.3 Member Function Documentation

5.1.3.1 converter_lat_lon()

```
static std::string GPSTracker::GPSData::converter_lat_lon (
    const std::string & string_numerica,
    const std::string & string_hemisf ) [inline], [static]
```

Função estática auxiliar para converter coordenadas NMEA (latitude/longitude) para graus decimais.

Parameters

<i>string_numerica</i>	String com a coordenada em formato NMEA (ex: "2257.34613").
<i>string_hemisf</i>	String com o hemisfério correspondente ("N", "S", "E", "W").

Returns

String coordenada em graus decimais (negativa para hemisférios Sul e Oeste).

Definition at line 123 of file [GPSTracker.hpp](#).

5.1.3.2 parsing()

```
bool GPSTracker::GPSData::parsing (
    int code_pattern,
    const std::vector< std::string > & data splitted ) [inline]
```

Setará os dados baseado no padrão de mensagem recebido.

Parameters

<i>code_pattern</i>	Código para informar que padrão de mensagem recebeu.
<i>data splitted</i>	Vetor de dados da mensagem recebida.

Returns

Retornará true caso seja bem sucedido. False, caso contrário.

Tradução de códigos:

- 0 == GPGBA

A partir do padrão de mensagem recebida, organizaremos o vetor com dados relevantes.

Definition at line 164 of file [GPSTracker.hpp](#).

5.1.3.3 to_csv()

```
std::string GPSTracker::GPSData::to_csv ( ) const [inline]
```

Retorna os dados armazenados em formato CSV.

Returns

std::string Linha CSV com os valores.

Definition at line 216 of file [GPSTracker.hpp](#).

5.1.4 Member Data Documentation

5.1.4.1 data

```
std::vector<std::string> GPSTracker::GPSData::data [private]
```

Organizaremos os dados neste vetor.

Definition at line 103 of file [GPSTracker.hpp](#).

5.1.4.2 pattern

```
std::string GPSTracker::GPSData::pattern [private]
```

Definition at line 102 of file [GPSTracker.hpp](#).

The documentation for this class was generated from the following file:

- [src/GPSTracker.hpp](#)

5.2 GPSTest.GPSTest Class Reference

Monitor que apresentará os dados do sensor.

Collaboration diagram for GPSTest.GPSTest:

GPSTest.GPSTest
+ time
+ traject
+ altitude
+ placeholder
+ zona_brasileira
+ sock
+ img_data
+ <code>__init__</code> (self, str host, int port, img="")
+ datetime conversion _utc_to_brasil(self, str time_utc)
+ None get_data_from _client(self)
+ None set_background (self)
+ None save_historic (self)
+ None mainloop(self)

Public Member Functions

- `__init__` (self, str host, int port, img="")
Construtor da Classe, inicializando atributos cruciais.
- datetime `conversion_utc_to_brasil` (self, str time_utc)
Converterá a string de time utc para string de time no fuso horário do Brasil.
- None `get_data_from_client` (self)
Verifica no buffer do socket o padrão de string setado.
- None `set_background` (self)
Setará uma imagem como background da aplicação.
- None `save_historic` (self)
Salvará dinamicamente os dados que já chegaram.
- None `mainloop` (self)
Responsável por executar as funcionalidades básicas.

Public Attributes

- [time](#)
- [traject](#)
- [altitude](#)
- [placeholder](#)
- [zona_brasileira](#)
- [sock](#)
- [img_data](#)

5.2.1 Detailed Description

Monitor que apresentará os dados do sensor.

Responsável por:

- Apresentar os dados do sensor em tempo real
- Possuir um histórico de dados
- Possibilitar o salvamento deles

Definition at line 15 of file [GPSPMonitor.py](#).

5.2.2 Constructor & Destructor Documentation

5.2.2.1 `__init__()`

```
GPSPMonitor.GPSPMonitor.__init__ (
    self,
    str host,
    int port,
    img = "" )
```

Construtor da Classe, inicializando atributos cruciais.

Parameters

<i>host</i>	IP da máquina deve receber os pacotes
<i>port</i>	Porta de Comunicação
<i>img</i>	caminho da imagem de background

Definition at line 25 of file [GPSPMonitor.py](#).

5.2.3 Member Function Documentation

5.2.3.1 `conversion_utc_to_brasil()`

```
datetime GPSPMonitor.GPSPMonitor.conversion_utc_to_brasil (
```

```
self,  
str time_utc )
```

Converterá a string de time utc para string de time no fuso horário do Brasil.

Parameters

<i>time_utc</i>	String time UTC
<i>String</i>	hora em Brasil

Definition at line 54 of file [GPSTest.py](#).

5.2.3.2 get_data_from_client()

```
None GPSTest.GPSTest.get_data_from_client (  
    self )
```

Verifica no buffer do socket o padrão de string setado.

Preenchendo as variáveis de estado.

Parameters

<i>host</i>	Endereço da Máquina Servidor
<i>port</i>	Porta de Entrada de Dados

Definition at line 72 of file [GPSTest.py](#).

5.2.3.3 mainloop()

```
None GPSTest.GPSTest.mainloop (  
    self )
```

Responsável por executar as funcionalidades básicas.

Dado o funcionamento do streamlit, executa as funções como se estivessem em loop. Verifique como o streamlit executa código python para mais informações.

Definition at line 144 of file [GPSTest.py](#).

5.2.3.4 save_historic()

```
None GPSTest.GPSTest.save_historic (  
    self )
```

Salvará dinamicamente os dados que já chegaram.

Definition at line 122 of file [GPSTest.py](#).

5.2.3.5 `set_background()`

```
None GPSMonitor.GPSMonitor.set_background (
    self )
```

Setará uma imagem como background da aplicação.

Definition at line 102 of file [GPSMonitor.py](#).

5.2.4 Member Data Documentation

5.2.4.1 `altitude`

```
GPSMonitor.GPSMonitor.altitude
```

Definition at line 36 of file [GPSMonitor.py](#).

5.2.4.2 `img_data`

```
GPSMonitor.GPSMonitor.img_data
```

Definition at line 44 of file [GPSMonitor.py](#).

5.2.4.3 `placeholder`

```
GPSMonitor.GPSMonitor.placeholder
```

Definition at line 37 of file [GPSMonitor.py](#).

5.2.4.4 `sock`

```
GPSMonitor.GPSMonitor.sock
```

Definition at line 40 of file [GPSMonitor.py](#).

5.2.4.5 `time`

```
GPSMonitor.GPSMonitor.time
```

Definition at line 34 of file [GPSMonitor.py](#).

5.2.4.6 `traject`

```
GPSMonitor.GPSMonitor.traject
```

Definition at line 35 of file [GPSMonitor.py](#).

5.2.4.7 zona_brasileira

GPSSim.GPSSim.zona_brasileira

Definition at line 39 of file [GPSSim.py](#).

The documentation for this class was generated from the following file:

- [src/GPSSim.py](#)

5.3 GPSSim Class Reference

Versão Simulada do Sensor GPS, gerando frases no padrão NMEA.

```
#include <GPSSim.hpp>
```

Collaboration diagram for GPSSim:

GPSSim
- int fd_pai - int fd_filho - char caminho_do_pseudo_terminal - std::thread worker - std::atomic< bool > is_exec - double lat - double lon - double alt - bool to_move
+ GPSSim(double latitude_inicial_graus, double longitude_inicial_graus, double altitude_metros, bool to_move) + ~GPSSim() + void init() + void stop() + std::string get_path_pseudo_term() const - void update() - void loop()

Classes

- class [NMEAGenerator](#)

Classe responsável por agrupar as funções geradoras de sentenças NMEA.

Public Member Functions

- [GPSSim](#) (double latitude_inicial_graus, double longitude_inicial_graus, double altitude_metros, bool to_move)

Construtor do [GPSSim](#).

- [~GPSSim](#) ()

Destrutor do [GPSSim](#).

- void [init](#) ()

Inicia a geração de frases no padrão NMEA em uma thread separada.

- void [stop](#) ()

Encerrará a geração de frases NMEA e aguardará a finalização da thread.

- std::string [get_path_pseudo_term](#) () const

Obtém o caminho do terminal que estamos executando de forma filial.

Private Member Functions

- void [update](#) ()

Responsável por prover alguma movimentação ao sensor simulado.

- void [loop](#) ()

Loop principal responsável pela geração e transmissão de dados simulados.

Private Attributes

- int [fd_pai](#) {0}
- int [fd_filho](#) {0}
- char [caminho_do_pseudo_terminal](#) [128]
- std::thread [worker](#)
- std::atomic< bool > [is_exec](#) {false}
- double [lat](#)
- double [lon](#)
- double [alt](#)
- bool [to_move](#) = false

5.3.1 Detailed Description

Versão Simulada do Sensor GPS, gerando frases no padrão NMEA.

Criará um par de pseudo-terminais (PTY) para simular o funcionamento do sensor. Intervaladamente, gera frases NMEA simuladas.

Definition at line 64 of file [GPSSim.hpp](#).

5.3.2 Constructor & Destructor Documentation

5.3.2.1 GPSSim()

```
GPSSim::GPSSim (
    double latitude_inicial_graus,
    double longitude_inicial_graus,
    double altitude_metros,
    bool to_move ) [inline]
```

Construtor do [GPSSim](#).

Inicializa alguns parâmetros de posição simulada e, criando os pseudo-terminais, configura-os para o padrão do módulo real.

Parameters

<i>latitude_inicial_graus</i>	Latitude inicial em graus decimais
<i>longitude_inicial_graus</i>	Longitude inicial em graus decimais
<i>altitude_metros</i>	Altitude inicial em metros, setada para 10.
<i>to_move</i>	Flag para provocarmos algum movimento

Definition at line [346](#) of file [GPSSim.hpp](#).

5.3.2.2 ~GPSSim()

```
GPSSim::~GPSSim ( ) [inline]
```

Destrutor do [GPSSim](#).

Chama a função [stop\(\)](#) e, após verificar existência de terminal Pai, fecha-o.

Definition at line [389](#) of file [GPSSim.hpp](#).

5.3.3 Member Function Documentation

5.3.3.1 get_path_pseudo_term()

```
std::string GPSSim::get_path_pseudo_term ( ) const [inline]
```

Obtém o caminho do terminal que estamos executando de forma filial.

Returns

String correspondendo ao caminho do dispositivo.

Definition at line [434](#) of file [GPSSim.hpp](#).

5.3.3.2 init()

```
void GPSSim::init ( ) [inline]
```

Inicia a geração de frases no padrão NMEA em uma thread separada.

O padrão NMEA é o protocolo padrão usado por módulos GPS, cada mensagem começa com \$ e termina com \r\n. Exemplo:

- \$origem,codificacao_usada,dados1,dados2,...*paridade

Definition at line 400 of file [GPSSim.hpp](#).

5.3.3.3 loop()

```
void GPSSim::loop ( ) [inline], [private]
```

Loop principal responsável pela geração e transmissão de dados simulados.

Esta função executa um laço contínuo enquanto o simulador estiver ativo (`_is_exec`). O loop termina automaticamente quando `_is_exec` é definido como falso. Também está presente a lógica de atualização de posição.

Definition at line 317 of file [GPSSim.hpp](#).

5.3.3.4 stop()

```
void GPSSim::stop ( ) [inline]
```

Encerrará a geração de frases NMEA e aguardará a finalização da thread.

Verifica a execução da thread e encerra-a caso exista. Força a finalização da thread geradora de mensagens.

Definition at line 417 of file [GPSSim.hpp](#).

5.3.3.5 update()

```
void GPSSim::update ( ) [inline], [private]
```

Responsável por prover alguma movimentação ao sensor simulado.

Definition at line 83 of file [GPSSim.hpp](#).

5.3.4 Member Data Documentation

5.3.4.1 alt

```
double GPSSim::alt [private]
```

Definition at line 76 of file [GPSSim.hpp](#).

5.3.4.2 caminho_do_pseudo_terminal

```
char GPSSim::caminho_do_pseudo_terminal[128] [private]
```

Definition at line 69 of file [GPSSim.hpp](#).

5.3.4.3 fd_filho

```
int GPSSim::fd_filho {0} [private]
```

Definition at line 68 of file [GPSSim.hpp](#).

5.3.4.4 fd_pai

```
int GPSSim::fd_pai {0} [private]
```

Definition at line 68 of file [GPSSim.hpp](#).

5.3.4.5 is_exec

```
std::atomic<bool> GPSSim::is_exec {false} [private]
```

Definition at line 73 of file [GPSSim.hpp](#).

5.3.4.6 lat

```
double GPSSim::lat [private]
```

Definition at line 76 of file [GPSSim.hpp](#).

5.3.4.7 lon

```
double GPSSim::lon [private]
```

Definition at line 76 of file [GPSSim.hpp](#).

5.3.4.8 to_move

```
bool GPSSim::to_move = false [private]
```

Definition at line 77 of file [GPSSim.hpp](#).

5.3.4.9 worker

```
std::thread GPSSim::worker [private]
```

Definition at line 72 of file [GPSSim.hpp](#).

The documentation for this class was generated from the following file:

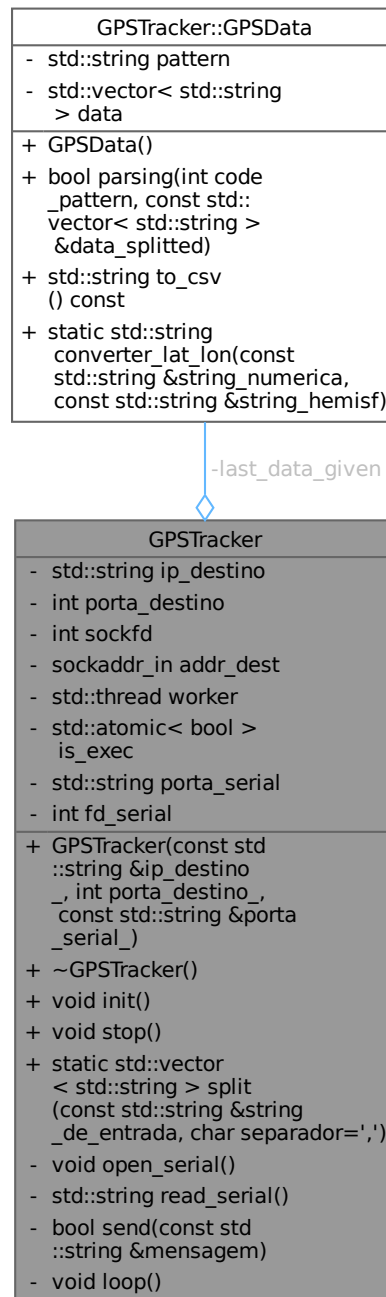
- [src/GPSSim.hpp](#)

5.4 GPSTracker Class Reference

Classe responsável por obter o tracking da carga.

```
#include <GPSTracker.hpp>
```

Collaboration diagram for GPSTracker:



Classes

- class [GPSData](#)

Classe responsável por representar os dados do GPS.

Public Member Functions

- [GPSTracker](#) (const std::string &ip_destino_, int porta_destino_, const std::string &porta_serial_)

Construtor da Classe.

- `~GPSTracker ()`

Destrutor da classe.

- `void init ()`

Inicializa a thread trabalhadora.

- `void stop ()`

Finaliza a thread de trabalho de forma segura.

Static Public Member Functions

- `static std::vector< std::string > split (const std::string &string_de_entrada, char separador=',')`

Função estática auxiliar para separar uma string em vetores de string.

Private Member Functions

- `void open_serial ()`

Abre e configura a porta serial para comunicação o sensor.

- `std::string read_serial ()`

Lê dados da porta serial até encontrar uma quebra de linha.

- `bool send (const std::string &mensagem)`

Envia uma string via socket UDP para um servidor.

- `void loop ()`

Executa o loop principal de leitura, interpretação e envio de dados via UDP.

Private Attributes

- `std::string ip_destino`
- `int porta_destino`
- `int sockfd`
- `sockaddr_in addr_dest {}`
- `std::thread worker`
- `std::atomic< bool > is_exec {false}`
- `GPSTData last_data_given`
- `std::string porta_serial`
- `int fd_serial = -1`

5.4.1 Detailed Description

Classe responsável por obter o tracking da carga.

Responsabilidades:

- Obter os dados do sensor NEO6MV2
- Interpretar esses dados, gerando informações
- Enviar as informações via socket UDP em formato CSV

Cada uma dessas responsabilidades está associada a um método da classe, respectivamente:

- [read_serial\(\)](#)
- [GPSTracker::parsing\(\)](#)
- [send\(\)](#)

Os quais estarão sendo repetidamente executados pela thread worker a fim de manter a continuidade de informações.

Não há necessidade de mais explicações, já que o fluxo de funcionamento é simples.

Definition at line 54 of file [GPSTracker.hpp](#).

5.4.2 Constructor & Destructor Documentation

5.4.2.1 GPSTracker()

```
GPSTracker::GPSTracker (
    const std::string & ip_destino_,
    int porta_destino_,
    const std::string & porta_serial_ ) [inline]
```

Construtor da Classe.

Parameters

<i>ip_destino_</i>	Endereço IP de destino.
<i>porta_↔destino_</i>	Porta UDP de destino
<i>porta_serial_↔_</i>	Caminho da porta serial

Inicializa a comunicação UDP e abre a comunicação serial.

Definition at line 499 of file [GPSTracker.hpp](#).

5.4.2.2 ~GPSTracker()

```
GPSTracker::~GPSTracker ( ) [inline]
```

Destrutor da classe.

Realiza a limpeza adequada dos recursos da classe, garantindo o término seguro das operações.

A ordem de operações é importante:

1. Interrompe a thread de execução
2. Fecha o socket de comunicação
3. Fecha a porta serial

Definition at line 534 of file [GPSTracker.hpp](#).

5.4.3 Member Function Documentation

5.4.3.1 init()

```
void GPSTracker::init ( ) [inline]
```

Inicializa a thread trabalhadora.

Garante que apenas uma única instância da thread de execução será iniciada, utilizando um flag atômico para controle de estado. Se a thread já estiver em execução, a função retorna imediatamente sem realizar nova inicialização. Ao iniciar, exibe uma mensagem colorida no terminal e cria uma thread worker que executa o loop principal de leitura e comunicação.

Definition at line 547 of file [GPSTracker.hpp](#).

5.4.3.2 loop()

```
void GPSTracker::loop ( ) [inline], [private]
```

Executa o loop principal de leitura, interpretação e envio de dados via UDP.

Esta função realiza continuamente a leitura de dados da porta serial, interpreta as mensagens GPS no formato GPGLL, armazena os dados processados e, se desejado, os exibe em formato CSV.

O loop executa enquanto a flag de execução estiver ativa, com uma pausa de 1 segundo entre cada iteração para evitar consumo excessivo de CPU.

Definition at line 441 of file [GPSTracker.hpp](#).

5.4.3.3 open_serial()

```
void GPSTracker::open_serial ( ) [inline], [private]
```

Abre e configura a porta serial para comunicação o sensor.

Estabelece a conexão serial utilizando a porta serial especificada. Todos os parâmetros necessários para uma comunicação estável com o dispositivo, incluindo velocidade, formato de dados e controle de fluxo são setados.

A porta é aberta em modo somente leitura (O_RDONLY) e em modo raw, no qual não há processamento adicional dos caracteres.

Aplicamos as seguintes configurações:

- 9600 bauds
- 8 bits de dados
- Sem paridade
- 1 bit de parada

Definition at line 305 of file [GPSTracker.hpp](#).

5.4.3.4 read_serial()

```
std::string GPSTracker::read_serial ( ) [inline], [private]
```

Lê dados da porta serial até encontrar uma quebra de linha.

- Caracteres de carriage return ('\r') são ignorados durante a leitura.
- A função termina quando encontra '\n' ou quando não há mais dados para ler.
- A leitura é feita caractere por caractere para garantir processamento correto dos dados do GPS que seguem protocolo NMEA.

Definition at line 374 of file [GPSTracker.hpp](#).

5.4.3.5 send()

```
bool GPSTracker::send (
    const std::string & mensagem ) [inline], [private]
```

Envia uma string via socket UDP para um servidor.

Parameters

<i>mensagem</i>	String a ser enviada.
-----------------	-----------------------

Returns

True se a mensagem foi enviada com sucesso. False, caso contrário.

Definition at line 412 of file [GPSTracker.hpp](#).

5.4.3.6 split()

```
static std::vector< std::string > GPSTracker::split (
    const std::string & string_de_entrada,
    char separador = ',' ) [inline], [static]
```

Função estática auxiliar para separar uma string em vetores de string.

Parameters

<i>string_de_entrada</i>	String que será fatiada.
<i>separador</i>	Caractere que será a flag de separação.

Similar ao método `split` do python, utiliza ',' como caractere separador default.

Definition at line 243 of file [GPSTracker.hpp](#).

5.4.3.7 stop()

```
void GPSTracker::stop ( ) [inline]
```

Finaliza a thread de trabalho de forma segura.

Esta função realiza o desligamento controlado da thread de trabalho. Primeiro, altera o flag de execução para falso usando operação atômica. Se a thread estiver joinable (executando), imprime uma mensagem de confirmação e realiza a operação de join para aguardar a finalização segura da thread.

Definition at line 566 of file [GPSTracker.hpp](#).

5.4.4 Member Data Documentation

5.4.4.1 addr_dest

```
sockaddr_in GPSTracker::addr_dest {} [private]
```

Definition at line 275 of file [GPSTracker.hpp](#).

5.4.4.2 fd_serial

```
int GPSTracker::fd_serial = -1 [private]
```

Definition at line 284 of file [GPSTracker.hpp](#).

5.4.4.3 ip_destino

```
std::string GPSTracker::ip_destino [private]
```

Definition at line 272 of file [GPSTracker.hpp](#).

5.4.4.4 is_exec

```
std::atomic<bool> GPSTracker::is_exec {false} [private]
```

Definition at line 279 of file [GPSTracker.hpp](#).

5.4.4.5 last_data_given

```
GPSTData GPSTracker::last_data_given [private]
```

Definition at line 282 of file [GPSTracker.hpp](#).

5.4.4.6 porta_destino

```
int GPSTracker::porta_destino [private]
```

Definition at line 273 of file [GPSTracker.hpp](#).

5.4.4.7 porta_serial

```
std::string GPSTracker::porta_serial [private]
```

Definition at line 283 of file [GPSTracker.hpp](#).

5.4.4.8 sockfd

```
int GPSTracker::sockfd [private]
```

Definition at line 274 of file [GPSTracker.hpp](#).

5.4.4.9 worker

```
std::thread GPSTracker::worker [private]
```

Definition at line 278 of file [GPSTracker.hpp](#).

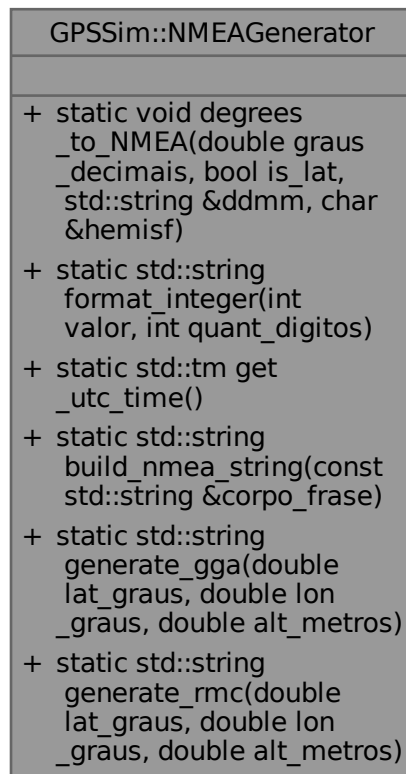
The documentation for this class was generated from the following file:

- [src/GPSTracker.hpp](#)

5.5 GPSSim::NMEAGenerator Class Reference

Classe responsável por agrupar as funções geradoras de sentenças NMEA.

Collaboration diagram for GPSSim::NMEAGenerator:



Static Public Member Functions

- static void [degrees_to_NMEA](#) (double graus_decimais, bool is_lat, std::string &ddmm, char &hemisf)
Converte graus decimais para formato NMEA de localização, (ddmm.mmmm).
- static std::string [format_integer](#) (int valor, int quant_digitos)
Formatada um número inteiro com dois dígitos, preenchendo com zero à esquerda.
- static std::tm [get_utc_time](#) ()
Obtém o tempo UTC atual, horário em Londres.
- static std::string [build_nmea_string](#) (const std::string &corpo_frase)
Finaliza uma sentença NMEA a partir do corpo da frase.
- static std::string [generate_gga](#) (double lat_graus, double lon_graus, double alt_metros)
Gera uma frase GGA (Global Positioning System Fix Data)
- static std::string [generate_rmc](#) (double lat_graus, double lon_graus, double alt_metros)
Gera uma frase GGA (Recommended Minimum Navigation Information)

5.5.1 Detailed Description

Classe responsável por agrupar as funções geradoras de sentenças NMEA.

Contém todas as funções que permitem a construção de sentenças NMEA a partir de informações precisas.

Definition at line 94 of file [GPSSim.hpp](#).

5.5.2 Member Function Documentation

5.5.2.1 build_nmea_string()

```
static std::string GPSSim::NMEAGenerator::build_nmea_string (
    const std::string & corpo_frase ) [inline], [static]
```

Finaliza uma sentença NMEA a partir do corpo da frase.

Calcula o valor de paridade (checksum) do corpo da frase fornecida, em seguida adiciona os delimitadores e flags no formato NMEA (prefixo '\$', sufixo '*', valor de paridade em hexadecimal e "\r\n").

Parameters

<i>corpo_frase</i>	Corpo da frase NMEA sem os indicadores iniciais ('\$') e finais ('*' e checksum).
--------------------	---

Returns

std::string Sentença NMEA completa, pronta para transmissão.

Definition at line 182 of file [GPSSim.hpp](#).

5.5.2.2 degrees_to_NMEA()

```
static void GPSSim::NMEAGenerator::degrees_to_NMEA (
    double graus_decimais,
    bool is_lat,
    std::string & ddm,
    char & hemisf ) [inline], [static]
```

Converte graus decimais para formato NMEA de localização, (ddmm.mmmm).

Parameters

	<i>graus_decimais</i>	Valor em graus decimais
	<i>is_lat</i>	Flag de eixo
out	<i>ddmm</i>	String com valor formatado em graus e minutos
out	<i>hemisf</i>	Caractere indicando Hemisfério

Definition at line 105 of file [GPSSim.hpp](#).

5.5.2.3 format_integer()

```
static std::string GPSSim::NMEAGenerator::format_integer (
    int valor,
    int quant_digitos ) [inline], [static]
```

Formatada um número inteiro com dois dígitos, preenchendo com zero à esquerda.

Parameters

<i>valor</i>	Número a ser formatado
<i>quant_digitos</i>	Quantidade de Dígitos presente

Returns

string formatada

Definition at line 144 of file [GPSSim.hpp](#).

5.5.2.4 generate_gga()

```
static std::string GPSSim::NMEAGenerator::generate_gga (  
    double lat_graus,  
    double lon_graus,  
    double alt_metros ) [inline], [static]
```

Gera uma frase GGA (Global Positioning System Fix Data)

Apesar de usar apenas valores de lat, long e alt, zera os demais valores.

Parameters

<i>lat_graus</i>	Latitude em graus decimais
<i>lon_graus</i>	Longitude em graus decimais
<i>alt_metros</i>	Altitude em metros

Returns

String correspondendo ao corpo de frase GGA

Definition at line 220 of file [GPSSim.hpp](#).

5.5.2.5 generate_rmc()

```
static std::string GPSSim::NMEAGenerator::generate_rmc (  
    double lat_graus,  
    double lon_graus,  
    double alt_metros ) [inline], [static]
```

Gera uma frase GGA (Recommended Minimum Navigation Information)

Apesar de usar apenas valores de lat, long e alt, zera os demais valores.

Parameters

<i>lat_graus</i>	Latitude em graus decimais
<i>lon_graus</i>	Longitude em graus decimais
<i>alt_metros</i>	Altitude em metros

Returns

String correspondendo ao corpo de frase GGA

Definition at line 268 of file [GPSSim.hpp](#).

5.5.2.6 get_utc_time()

```
static std::tm GPSSim::NMEAGenerator::get_utc_time ( ) [inline], [static]
```

Obtém o tempo UTC atual, horário em Londres.

Returns

Struct std::tm contendo o tempo em UTC

Definition at line 161 of file [GPSSim.hpp](#).

The documentation for this class was generated from the following file:

- [src/GPSSim.hpp](#)

Chapter 6

File Documentation

6.1 src/debug.cpp File Reference

Responsável por prover ferramentas de debug.

```
#include <iostream>
#include "GPSTracker.hpp"
#include "GPSSim.hpp"
Include dependency graph for debug.cpp:
```



Functions

- int [main](#) (int argc, char *argv[])

6.1.1 Detailed Description

Responsável por prover ferramentas de debug.

Já que a aplicação deve ser executada dentro da placa, não conseguiríamos executá-la no DeskTop. Para tanto, fez-se necessário o desenvolvimento de ferramentas que possibilitam a debugação de nosso código.

Definition in file [debug.cpp](#).

6.1.2 Function Documentation

6.1.2.1 main()

```
int main (
    int argc,
    char * argv[] )
```

Definition at line 15 of file [debug.cpp](#).

6.2 debug.cpp

[Go to the documentation of this file.](#)

```

00001
00009 #include <iostream>
00010 #include "GPSTracker.hpp"
00011 #include "GPSSim.hpp"
00012
00013 // IME - -22.9559, -43.1659
00014
00015 int main(
00016     int argc,
00017     char* argv[]
00018 ){
00019     if(argc == 1){
00020
00021         std::cout << "Falta informar o IP e a PORTA de destino." << std::endl;
00022         return -1;
00023     }
00024     else if(argc == 2){
00025
00026         std::cout << "Falta informar a PORTA de destino." << std::endl;
00027         return -1;
00028     }
00029     else if(argc > 3){
00030
00031         std::cout << "Há argumentos inválidos, informe apenas IP e PORTA de destino." << std::endl;
00032         return -1;
00033     }
00034
00035     // Inicializamos o módulo gps simulado
00036     GPSSim gps_module(
00037         -22.9559,
00038         -43.1659,
00039         32.0,
00040         true
00041     );
00042     std::cout << "Executando simulator_gps_module em: "
00043         << gps_module.get_path_pseudo_term()
00044         << "\n";
00045     gps_module.init();
00046
00047     // Apenas espera para não encher o buffer
00048     std::this_thread::sleep_for(std::chrono::seconds(1));
00049
00050     GPSTracker sensor(
00051         argv[1],
00052         std::stoi(argv[2]),
00053         gps_module.get_path_pseudo_term()
00054     );
00055     sensor.init();
00056
00057     std::this_thread::sleep_for(std::chrono::seconds(60));
00058
00059     sensor.stop();
00060     gps_module.stop();
00061     return 0;
00062 }

```

6.3 src/GPSMonitor.py File Reference

Implementação da interface de visualização de dados.

Classes

- class [GPSMonitor.GPSMonitor](#)
Monitor que apresentará os dados do sensor.

Namespaces

- namespace [GPSMonitor](#)

Variables

- str `GPSMonitor.gps_monitor` = `GPSMonitor`(img="src/antena.jpg", host="127.0.0.1", port=5000) if "self" not in st.session_state else st.session_state["self"]

6.3.1 Detailed Description

Implementação da interface de visualização de dados.

Definition in file [GPSMonitor.py](#).

6.4 GPSMonitor.py

[Go to the documentation of this file.](#)

```
00001 """
00002 @file GPSMonitor.py
00003 @brief Implementação da interface de visualização de dados
00004 """
00005
00006 import base64
00007 import streamlit as st
00008 from streamlit_autorefresh import st_autorefresh
00009 import pandas as pd
00010 import socket
00011 from datetime import datetime
00012 import pytz
00013 import pydeck as pdk
00014
00015 class GPSMonitor:
00016     """
00017     @brief Monitor que apresentará os dados do sensor
00018     @details
00019     Responsável por:
00020     - Apresentar os dados do sensor em tempo real
00021     - Possuir um histórico de dados
00022     - Possibilitar o salvamento deles
00023     """
00024
00025     def __init__(self, host: str, port: int, img=""):
00026         """
00027         @brief Construtor da Classe, inicializando atributos cruciais.
00028         @param host IP da máquina deve receber os pacotes
00029         @param port Porta de Comunicação
00030         @param img caminho da imagem de background
00031         """
00032
00033         st.session_state["self"] = self
00034         self.time = []
00035         self.traject = []
00036         self.altitude = []
00037         self.placeholder = st.empty() # Apenas para conseguirmos renderizar o mapa corretamente
00038
00039         self.zona_brasileira = pytz.timezone("America/Sao_Paulo")
00040         self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
00041         self.sock.bind((host, port))
00042         self.sock.settimeout(0.5)
00043
00044         self.img_data = None
00045         if img:
00046             with open(
00047                 img,
00048                 "rb"
00049             ) as f:
00050                 data = f.read()
00051
00052                 self.img_data = base64.b64encode(data).decode()
00053
00054     def conversion_utc_to_brasil(
00055         self,
00056         time_utc: str
00057     ) -> datetime:
00058         """
00059         @brief Converterá a string de time utc para string de time no fuso horário do Brasil
00060         @param time_utc: String time UTC
```

```

00061         @param String hora em Brasil
00062         """
00063
00064         hora_utc = datetime.strptime(time_utc, "%H%M%S").time()
00065         data_hora_utc = datetime.combine(datetime.now().date(), hora_utc)
00066
00067         data_hora_utc = pytz.UTC.localize(data_hora_utc)
00068         data_hora_brasil = data_hora_utc.astimezone(self.zona_brasileira)
00069
00070         return data_hora_brasil
00071
00072     def get_data_from_client(
00073         self
00074     ) -> None:
00075         """
00076         @brief Verifica no buffer do socket o padrão de string setado. Preenchendo as variáveis de
00077 estado.
00078         @param host: Endereço da Máquina Servidor
00079         @param port: Porta de Entrada de Dados
00080         """
00081         try:
00082             data, _ = self.sock.recvfrom(1024)
00083             numeros = data.decode().strip().split(",")
00084             if numeros:
00085                 # Obtendo informações de horário
00086                 self.time.append(
00087                     {"time": self.conversion_utc_to_brasil(numeros[0][-3:])}
00088                 )
00089
00090                 # Obtendo informações de geolocalização
00091                 self.traject.append(
00092                     {"latitude": float(numeros[1]), "longitude": float(numeros[2])}
00093                 )
00094
00095                 self.altitude.append(
00096                     {"altitude": float(numeros[3])}
00097                 )
00098
00099             except TimeoutError:
00100                 print("Esperando...")
00101
00102     def set_background(
00103         self
00104     ) -> None:
00105         """
00106         @brief Setará uma imagem como background da aplicação
00107         """
00108         st.markdown(
00109             f"""
00110             <style>
00111             .stApp {{
00112                 background-image: url("data:image/jpg;base64,{self.img_data}");
00113                 background-size: cover;
00114                 background-position: center;
00115                 background-repeat: no-repeat;
00116             }}
00117             </style>
00118             """
00119             ,
00120             unsafe_allow_html=True
00121         )
00122
00123     def save_historic(self) -> None:
00124         """
00125         @brief Salvará dinamicamente os dados que já chegaram.
00126         """
00127         with open(
00128             "historico.csv",
00129             "w"
00130         ) as file:
00131             file.write(
00132                 pd.DataFrame(
00133                     self.time
00134                 ).join(
00135                     pd.DataFrame(
00136                         self.traject
00137                     ).join(
00138                         pd.DataFrame(
00139                             self.altitude
00140                         )
00141                     )
00142                 ).to_csv()
00143             )
00144
00145     def mainloop(self) -> None:
00146         """
00147         @brief Responsável por executar as funcionalidades básicas.

```

```

00147         @details
00148         Dado o funcionamento do streamlit, executa as funções como se estivessem em loop.
00149         Verifique como o streamlit executa código python para mais informações.
00150         """
00151
00152         self.get_data_from_client()
00153
00154         st.set_page_config(
00155             layout="wide"
00156         )
00157         st.markdown('<h1 style="color: white;">Paz para Você, Proteção para Sua Carga. Tecnologia a
Seu Favor.</h1>', unsafe_allow_html=True)
00158         if self.traject: # Apenas verificando se não está vazia
00159
00160             df = pd.DataFrame(
00161                 self.traject
00162             )
00163
00164             lat_center = df["latitude"].mean()
00165             lon_center = df["longitude"].mean()
00166
00167             layer = pdk.Layer(
00168                 "ScatterplotLayer",
00169                 data=df,
00170                 get_position='[longitude, latitude]',
00171                 get_color='[235, 140, 52, 255]',
00172                 get_radius=50,
00173                 pickable=True,
00174                 filled=True
00175             )
00176
00177             view_state = pdk.ViewState(
00178                 latitude=lat_center,
00179                 longitude=lon_center,
00180                 zoom=13, # ajuste conforme necessário
00181                 pitch=0
00182             )
00183
00184             st.pydeck_chart(
00185                 pdk.Deck(
00186                     layers=[layer], initial_view_state=view_state
00187                 )
00188             )
00189
00190             if st.button("Salvar Histórico"):
00191                 self.save_historic()
00192                 st.success("Dados Salvos!")
00193
00194             else:
00195                 st.write("Esperando dados chegarem...")
00196
00197             if self.img_data:
00198                 self.set_background()
00199             st_autorefresh(interval=1000, limit=None) # Para recarregarmos a aplicação
00200
00201
00202
00203 if __name__ == '__main__':
00204
00205     gps_monitor = GPSSimulator(img="src/antena.jpg", host="127.0.0.1", port=5000) if "self" not in
st.session_state else st.session_state["self"]
00206     gps_monitor.mainloop()

```

6.5 src/GPSSim.hpp File Reference

Implementação da Classe Simuladora do GPS6MV2.

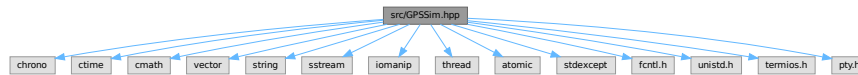
```

#include <chrono>
#include <ctime>
#include <cmath>
#include <vector>
#include <string>
#include <sstream>
#include <iomanip>
#include <thread>
#include <atomic>

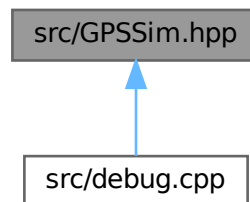
```

```
#include <stdexcept>
#include <fcntl.h>
#include <unistd.h>
#include <termios.h>
#include <pty.h>
```

Include dependency graph for GPSSim.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [GPSSim](#)
Versão Simulada do Sensor GPS, gerando frases no padrão NMEA.
- class [GPSSim::NMEAGenerator](#)
Classe responsável por agrupar as funções geradoras de sentenças NMEA.

6.5.1 Detailed Description

Implementação da Classe Simuladora do GPS6MV2.

Supondo que o módulo GPS6MV2 não esteja disponível, a classe implementada neste arquivo tem como objetivo simular todas as funcionalidades do mesmo.

Definition in file [GPSSim.hpp](#).

6.6 GPSSim.hpp

[Go to the documentation of this file.](#)

```

00001
00008 #ifndef GPSSim_HPP
00009 #define GPSSim_HPP
00010
00011 //-----
00012
00013 // Para manipulações de Tempo e de Data
00014 #include <chrono>
00015 #include <ctime>
00016
00017 #include <cmath>
00018 #include <vector>
00019
00020 #include <string>
00021 #include <sstream>
00022 #include <iomanip>
00023
00024 // Para threads e sincronizações
00025 #include <thread>
00026 #include <atomic>
00027
00028 // Para tratamento de erros
00029 #include <stdexcept>
00030
00031 // As seguintes bibliotecas possuem relevância superior
00032 // Por se tratarem de bibliotecas C, utilizaremos o padrão de `::` para explicitar
00033 // que algumas funções advém delas.
00034 /*
00035 Fornece constantes e funções de controle de descritores de arquivos,
00036 operações de I/O de baixo nível e manipulação de flags de arquivos.
00037 */
00038 #include <fcntl.h>
00039 /*
00040 Fornece acesso a chamadas do OS de baixo nível, incluindo manipulação
00041 de processos, I/O de arquivos, controle de descritores e operações do
00042 sistema de arquivos.
00043 */
00044 #include <unistd.h>
00045 /*
00046 Fornece estruturas e funções para configurar a comunicação
00047 em sistemas Unix. Ele permite o controle detalhado sobre interfaces
00048 de terminal (TTY)
00049 */
00050 #include <termios.h>
00051 /*
00052 Fornece funções para criação e manipulação de pseudo-terminais (PTYs),
00053 um mecanismo essencial em sistemas Unix para emular terminais virtuais.
00054 */
00055 #include <pty.h>
00056
00064 class GPSSim {
00065 private:
00066
00067     // Informações ligadas ao terminal
00068     int fd_pai{0}, fd_filho{0};
00069     char caminho_do_pseudo_terminal[128];
00070
00071     // Thread de Execução Paralela e Flag de Controle
00072     std::thread worker;
00073     std::atomic<bool> is_exec{false};
00074
00075     // Informações de Localização
00076     double lat, lon, alt;
00077     bool to_move = false;
00078
00082     void
00083     update(){
00084         lat -= 0.0001;
00085         lon -= 0.0001;
00086     }
00087
00094     class NMEAGenerator {
00095     public:
00096
00104         static void
00105         degrees_to_NMEA(
00106             double graus_decimais,
00107             bool is_lat,
00108             std::string& ddmm,
00109             char& hemisf
00110         ){
00111             hemisf = (is_lat) ? (

```

```

00112             ( graus_decimais >= 0 ) ? 'N' : 'S'
00113             ) :
00114             (
00115             ( graus_decimais >= 0 ) ? 'E' : 'W'
00116             );
00117
00118     double valor_abs = std::fabs(graus_decimais);
00119     int graus = static_cast<int>(std::floor(valor_abs));
00120     double min = (valor_abs - graus) * 60.0;
00121
00122     // Não utilizamos apenas a função de formatar_inteiro, pois
00123     // utilizaremos o oss em seguida.
00124     std::ostringstream oss;
00125     oss << std::setfill('0')
00126         << std::setw(is_lat ? 2 : 3)
00127         << graus
00128         << std::fixed
00129         << std::setprecision(4)
00130         << std::setw(7)
00131         << std::right
00132         << min;
00133
00134     ddmn = oss.str();
00135 }
00136
00143 static std::string
00144 format_integer(
00145     int valor,
00146     int quant_digitos
00147 ){
00148
00149     std::ostringstream oss;
00150     oss << std::setw(quant_digitos)
00151         << std::setfill('0')
00152         << valor;
00153     return oss.str();
00154 }
00155
00160 static std::tm
00161 get_utc_time(){
00162
00163     using namespace std::chrono;
00164     auto tempo_atual = system_clock::to_time_t(system_clock::now());
00165     std::tm tempo_utc{};
00166     gmtime_r(&tempo_atual, &tempo_utc);
00167     return tempo_utc;
00168 }
00169
00181 static std::string
00182 build_nmea_string(
00183     const std::string& corpo_frase
00184 ){
00185
00186     uint8_t paridade = 0;
00187     for(
00188         const char& caract : corpo_frase
00189     ){
00190
00191         paridade ^= (uint8_t)caract;
00192     }
00193
00194     std::ostringstream oss;
00195     oss << '$'
00196         << corpo_frase
00197         << '*'
00198         << std::uppercase
00199         << std::hex
00200         << std::setw(2)
00201         << std::setfill('0')
00202         << (int)paridade
00203         << "\r\n";
00204
00205     return oss.str();
00206 }
00207
00219 static std::string
00220 generate_gga(
00221     double lat_graus,
00222     double lon_graus,
00223     double alt_metros
00224 ){
00225
00226     auto tempo_utc = get_utc_time();
00227     std::string lat_nmea, lon_nmea;
00228     char hemisferio_lat, hemisferio_lon;
00229
00230     degrees_to_NMEA(

```



```

00231         lat_graus,
00232         true,
00233         lat_nmea,
00234         hemisferio_lat
00235     );
00236     degrees_to_NMEA(
00237         lon_graus,
00238         false,
00239         lon_nmea,
00240         hemisferio_lon
00241     );
00242
00243     // Formato: hhmmss.ss,lat,N/S,lon,E/W,qualidade,satelites,HDOP,altitude,M,...
00244     std::ostringstream oss;
00245     oss << "GPGGA," << format_integer(tempo_utc.tm_hour, 2)
00246         << format_integer(tempo_utc.tm_min, 2)
00247         << format_integer(tempo_utc.tm_sec, 2) << ".00,"
00248         << lat_nmea << "," << hemisferio_lat << ","
00249         << lon_nmea << "," << hemisferio_lon << ",1,"
00250         << "-1 << "," << std::fixed << std::setprecision(1) << "-1 << ","
00251         << std::fixed << std::setprecision(1) << alt_metros << ",M,0.0,M,";
00252
00253     return build_nmea_string(oss.str());
00254 }
00255
00267 static std::string
00268 generate_rmc(
00269     double lat_graus,
00270     double lon_graus,
00271     double alt_metros
00272 ){
00273
00274     auto tempo_utc = get_utc_time();
00275     std::string lat_nmea, lon_nmea;
00276     char hemisferio_lat, hemisferio_lon;
00277
00278     degrees_to_NMEA(
00279         lat_graus,
00280         true,
00281         lat_nmea,
00282         hemisferio_lat
00283     );
00284     degrees_to_NMEA(
00285         lon_graus,
00286         false,
00287         lon_nmea,
00288         hemisferio_lon
00289     );
00290
00291     // Formato: hhmmss.ss,A,lat,N/S,lon,E/W,velocidade,curso,data,,
00292     std::ostringstream oss;
00293     oss << "GPRMC,"
00294         << format_integer(tempo_utc.tm_hour, 2)
00295         << format_integer(tempo_utc.tm_min, 2)
00296         << format_integer(tempo_utc.tm_sec, 2) << ".00,A,"
00297         << lat_nmea << "," << hemisferio_lat << ","
00298         << lon_nmea << "," << hemisferio_lon << ","
00299         << std::fixed << std::setprecision(2) << "-1 << ",0.00,"
00300         << format_integer(tempo_utc.tm_mday, 2)
00301         << format_integer(tempo_utc.tm_mon + 1, 2)
00302         << format_integer((tempo_utc.tm_year + 1900) % 100, 2)
00303         << ",,A,";
00304
00305     return build_nmea_string(oss.str());
00306 }
00307 };
00308
00316 void
00317 loop(){
00318
00319     while (is_exec){
00320
00321         if(to_move){ update(); }
00322         std::string saida = NMEAGenerator::generate_gga(lat, lon, alt);
00323         std::cout << "\033[7mGPS6MV2 Simulado Emitindo:\033[0m " << saida << std::endl;
00324
00325         // Imprimimos no terminal serial
00326         (void)!::write(fd_pai, saida.data(), saida.size());
00327
00328         // Aguarda o próximo ciclo
00329         std::this_thread::sleep_for(std::chrono::seconds(1));
00330     }
00331 }
00332
00333 public:
00334
00346 GPSSim(

```

```

00347         double latitude_inicial_graus,
00348         double longitude_inicial_graus,
00349         double altitude_metros,
00350         bool to_move
00351     ) : lat(latitude_inicial_graus),
00352         lon(longitude_inicial_graus),
00353         alt(altitude_metros),
00354         to_move(to_move)
00355     {
00356
00357         // Cria o par de pseudo-terminais
00358         if(
00359             ::openpty( &fd_pai, &fd_filho, caminho_do_pseudo_terminal, nullptr, nullptr ) != 0
00360         ){
00361             throw std::runtime_error("Falha ao criar pseudo-terminal");
00362         }
00363
00364         // Configura o terminal filho para simular o módulo real (9600 8N1)
00365         termios config_com{}; // Cria a estrutura vazia
00366         ::tcgetattr(fd_filho, &config_com); // Lê as configurações atuais e armazena na struct
00367         ::cfsetispeed(&config_com, B9600); // Definimos velocidade de entrada e de saída
00368         ::cfsetospeed(&config_com, B9600); // Essa constante está presente dentro do termios.h
00369         // Diversas operações bits a bits
00370         config_com.c_cflag = (config_com.c_cflag & ~CSIZE) | CS8;
00371         config_com.c_cflag |= (CLOCAL | CREAD);
00372         config_com.c_cflag &= ~(PARENB | CSTOPB);
00373         config_com.c_iflag = IGNPAR;
00374         config_com.c_oflag = 0;
00375         config_com.c_lflag = 0;
00376         tcsetattr(fd_filho, TCSANOW, &config_com); // Aplicamos as configurações
00377
00378         // Fecha o filho - será aberto pelo usuário no caminho correto
00379         // Mantemos o Pai aberto para procedimentos posteriores
00380         ::close(fd_filho);
00381     }
00382
00383 ~GPSSim(){ stop(); if( fd_pai >= 0 ){ ::close(fd_pai); } }
00384
00385 void
00386 init(){
00387     if( is_exec.exchange(true) ){ return; }
00388
00389     worker = std::thread(
00390         [this]{ loop(); }
00391     );
00392 }
00393
00394 void
00395 stop(){
00396     if( !is_exec.exchange(false) ){ return; }
00397
00398     if(
00399         worker.joinable()
00400     ){
00401         worker.join();
00402     }
00403 }
00404
00405 std::string
00406 get_path_pseudo_term() const { return std::string(caminho_do_pseudo_terminal); }
00407 };
00408
00409 #endif // GPSSim_HPP

```

6.7 src/GPSTracker.hpp File Reference

Implementação da solução embarcada.

```

#include <string>
#include <vector>
#include <sstream>
#include <iomanip>
#include <iostream>
#include <cstring>

```

```

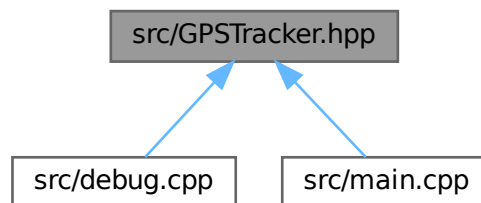
#include <chrono>
#include <cmath>
#include <ctime>
#include <thread>
#include <atomic>
#include <stdexcept>
#include <fcntl.h>
#include <termios.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>

```

Include dependency graph for GPSTracker.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [GPSTracker](#)
Classe responsável por obter o tracking da carga.
- class [GPSTracker::GPSData](#)
Classe responsável por representar os dados do GPS.

6.7.1 Detailed Description

Implementação da solução embarcada.

Definition in file [GPSTracker.hpp](#).

6.8 GPSTracker.hpp

[Go to the documentation of this file.](#)

```

00001
00005 #ifndef GPSTracker_HPP
00006 #define GPSTracker_HPP
00007
00008
00009 //-----
00010 #include <string>
00011 #include <vector>
00012 #include <sstream>
00013 #include <iomanip>
00014 #include <iostream>
00015 #include <cstring>
00016
00017 #include <chrono>
00018 #include <cmath>
00019 #include <ctime>
00020
00021 #include <thread>
00022 #include <atomic>
00023
00024 #include <stdexcept>
00025
00026 // Específicos de Sistemas Linux
00027 #include <fcntl.h>
00028 #include <termios.h>
00029 #include <unistd.h>
00030 #include <sys/socket.h>
00031 #include <arpa/inet.h>
00032 #include <netinet/in.h>
00033
00054 class GPSTracker {
00055 public:
00056
00099     class GPSData {
00100     private:
00101
00102         std::string pattern;
00103         std::vector<std::string> data;
00104
00105     public:
00106
00114         GPSData() { data.reserve(4); }
00115
00122         static std::string
00123         converter_lat_lon(
00124             const std::string& string_numerica,
00125             const std::string& string_hemisf
00126         ) {
00127
00128             if( string_numerica.empty() ){ return ""; }
00129
00130             double valor_cru = 0;
00131             try {
00132
00133                 valor_cru = std::stod(string_numerica);
00134             }
00135             catch (std::invalid_argument&) {
00136
00137                 std::cout << "\033[1;31mErro dentro de converter_lat_lon, valor inválido para stod:
00138 \033[0m"
00139                 << string_numerica
00140                 << std::endl;
00141
00142                 // Parsing dos valores
00143                 double graus = floor(valor_cru / 100);
00144                 double minutos = valor_cru - graus * 100;
00145                 double coordenada = (graus + minutos / 60.0) * ( (string_hemisf == "S" || string_hemisf ==
00146 "W" ) ? -1 : 1 );
00147
00148                 return std::to_string(coordenada);
00149             }
00163         bool
00164         parsing(
00165             int code_pattern,
00166             const std::vector<std::string>& data_splitted
00167         ) {
00168
00169             data.clear(); // Garantimos que está limpo.
00170
00171             if(

```

```

00172         code_pattern == 0
00173     ){
00174         // Em gga, os dados corretos estão em:
00175
00176         int idx_data_useful[] = {
00177             1, // Horário UTC
00178             2, // Latitude em NMEA
00179             4, // Longitude em NMEA
00180             9 // Altitude
00181         };
00182
00183         for(
00184             const auto& idx : idx_data_useful
00185         ){
00186
00187             if( idx == 2 || idx == 4 ){
00188                 data.emplace_back(
00189                     std::move(converter_lat_lon(
00190                         dataSplitted[idx],
00191                         dataSplitted[idx + 1]
00192                     ))
00193                 );
00194             }
00195             else{
00196                 // Então basta adicionar
00197                 data.emplace_back(
00198                     // Método bizurado para não realizarmos cópias
00199                     std::move(dataSplitted[idx])
00200                 );
00201             }
00202         }
00203
00204         return true;
00205     }
00206     // ... podemos escalar para novos padrões de mensagem
00207
00208     return false;
00209 }
00210
00215 std::string
00216 to_csv() const {
00217
00218     std::ostringstream oss;
00219     for (
00220         int i = 0;
00221         i < 4;
00222         i++
00223     ){
00224         if(i > 0){ oss << ","; }
00225
00226         oss << data[i];
00227     }
00228
00229     oss << "\n";
00230
00231     return oss.str();
00232 }
00233 };
00234
00242 static std::vector<std::string>
00243 split(
00244     const std::string& string_de_entrada,
00245     char separador=',',
00246 ){
00247
00248     std::vector<std::string> elementos;
00249     std::stringstream ss(string_de_entrada);
00250
00251     std::string elemento_individual;
00252     while(
00253         std::getline(
00254             ss,
00255             elemento_individual,
00256             separador
00257         )
00258     ){
00259         if(
00260             !elemento_individual.empty()
00261         ){
00262             elementos.push_back(elemento_individual);
00263         }
00264     }
00265
00266     return elementos;
00267 }
00268
00269

```

```

00270 private:
00271     // Relacionadas ao Envio UDP
00272     std::string ip_destino;
00273     int porta_destino;
00274     int sockfd;
00275     sockaddr_in addr_dest{};
00276
00277     // Relacionados ao fluxo de funcionamento
00278     std::thread worker;
00279     std::atomic<bool> is_exec{false};
00280
00281     // Relacionados à comunicação com o sensor
00282     GPSData last_data_given;
00283     std::string porta_serial;
00284     int fd_serial = -1;
00285
00286     void
00305     open_serial(){
00306
00307         fd_serial = ::open(
00308             porta_serial.c_str(),
00309             // O_RDONLY: garante apenas leitura
00310             // O_NOCTTY: impede que a porta se torne o terminal controlador do
00311             processo // O_SYNC: garante que as operações de escrita sejam completadas
00312             fisicamente O_RDONLY | O_NOCTTY | O_SYNC
00313             );
00314
00315         // Confirmação de sucesso
00316         if( fd_serial < 0 ){ throw std::runtime_error("\033[1;31mErro ao abrir porta serial do
GPS\033[0m"); }
00317
00318         // Struct para armazenarmos os parâmetros da comunicação serial.
00319         termios tty{};
00320         if(
00321             ::tcgetattr(
00322                 fd_serial,
00323                 &tty
00324             ) != 0
00325         ){ throw std::runtime_error("\033[1;31mErro ao tentar configurar a porta serial,
especificamente, tcgetattr\033[0m"); }
00326
00327         // Setamos velocidade
00328         ::cfsetospeed(&tty, B9600);
00329         ::cfsetispeed(&tty, B9600);
00330
00331         // Modo raw para não haver processamento por parte do sensor.
00332         ::cfmakeraw(&tty);
00333
00334         // Configuração 8N1
00335         tty.c_cflag &= ~CSIZE;
00336         tty.c_cflag |= CS8; // 8 bits
00337         tty.c_cflag &= ~PARENB; // sem paridade
00338         tty.c_cflag &= ~CSTOPB; // 1 stop bit
00339         tty.c_cflag &= ~CRTSCTS; // sem controle de fluxo por hardware
00340
00341         // Habilita leitura na porta
00342         tty.c_cflag |= (CLOCAL | CREAD);
00343
00344         // Não encerra a comunicação serial quando 'desligamos'
00345         tty.c_cflag &= ~HUPCL;
00346
00347         tty.c_iflag &= ~IGNBRK;
00348         tty.c_iflag &= ~(IXON | IXOFF | IXANY); // sem controle de fluxo por software
00349         tty.c_lflag = 0; // sem canonical mode, echo, signals
00350         tty.c_oflag = 0;
00351
00352         tty.c_cc[VMIN] = 1; // lê pelo menos 1 caractere
00353         tty.c_cc[VTIME] = 1; // timeout em décimos de segundo (0.1s)
00354
00355         if(
00356             ::tcsetattr(
00357                 fd_serial,
00358                 TCSANOW,
00359                 &tty
00360             ) != 0
00361         ){ throw std::runtime_error("\033[1;31mErro ao tentar setar configurações na comunicação
serial, especificamente, tcsetattr\033[0m"); }
00362     }
00363
00373     std::string
00374     read_serial(){
00375
00376         std::string buffer;
00377         char caract = '\0';
00378

```

```

00379         while(
00380             true
00381         ){
00382             int n = read(
00383                 fd_serial,
00384                 &caract,
00385                 1
00386             );
00387
00388             // Confirmação de sucesso.
00389             if(n > 0){
00390
00391                 // Então é caractere válido.
00392                 if(caract == '\n'){ break; }
00393                 if(caract != '\r'){ buffer += caract; } // Ignoramos o \r
00394
00395             }
00396             else if(n == 0){ break; } // Nada a ser lido
00397             else{
00398
00399                 throw std::runtime_error("\033[1;31mErro na leitura\033[0m");
00400             }
00401         }
00402
00403         return buffer;
00404     }
00405
00411     bool
00412     send(
00413         const std::string& mensagem
00414     ){
00415
00416         ssize_t bytes = ::sendto(
00417             sockfd,
00418             mensagem.c_str(),
00419             mensagem.size(),
00420             0,
00421             reinterpret_cast<struct sockaddr*>(&addr_dest),
00422             sizeof(addr_dest)
00423         );
00424
00425         if(bytes < 0){ std::cout << "Erro ao enviar" << std::endl; return false; }
00426         return true;
00427     }
00428
00429     void
00441     loop(){
00442
00443         bool parsed = false; // Apenas uma flag para sabermos se houve interpretação
00444         while(
00445             is_exec
00446         ){
00447
00448             std::string mensagem = read_serial();
00449
00450             if(mensagem.empty()){ std::cout << "Nada a ser lido..." << std::endl; }
00451             else{
00452
00453                 std::cout << "Recebendo: " << mensagem << std::endl;
00454
00455                 if( mensagem.find("GGA") != std::string::npos ){
00456
00457                     last_data_given.parsing(0, split(mensagem));
00458                     parsed = true;
00459                 }
00460                 // ... para escalarmos novos padrões de mensagem
00461                 else{
00462
00463                 }
00464
00465                 if(
00466                     parsed
00467                 ){
00468
00469                     std::string mensagem = last_data_given.to_csv();
00470
00471                     std::cout << "Interpretando: \033[7m"
00472                         << mensagem
00473                         << "\033[0m"
00474                         << std::endl;
00475
00476                     send(
00477                         mensagem
00478                     );
00479                     parsed = false;
00480                     printf("\n");

```

```

00481         }
00482     }
00483
00484     std::this_thread::sleep_for(std::chrono::seconds(1));
00485 }
00486 }
00487
00488 public:
00489
00490 GPSTracker(
00491     const std::string& ip_destino_,
00492     int porta_destino_,
00493     const std::string& porta_serial_
00494 ) : ip_destino(ip_destino_),
00495     porta_destino(porta_destino_),
00496     porta_serial(porta_serial_)
00497 {
00498     // As seguintes definições existentes para a comunicação UDP.
00499     sockfd = ::socket(AF_INET, SOCK_DGRAM, 0);
00500     if( sockfd < 0 ){
00501         throw std::runtime_error("Erro ao criar socket UDP");
00502     }
00503
00504     addr_dest.sin_family = AF_INET;
00505     addr_dest.sin_port = ::htons(porta_destino);
00506     addr_dest.sin_addr.s_addr = ::inet_addr(ip_destino.c_str());
00507
00508     open_serial();
00509 }
00510
00511 ~GPSTracker() { stop(); if( sockfd >= 0 ) { ::close(sockfd); } if( fd_serial >= 0 ) {
00512 ::close(fd_serial); } }
00513
00514 void
00515 init(){
00516     if( is_exec.exchange(true) ){ return; }
00517
00518     std::cout << "\033[1;32mIniciando Thread de Leitura...\033[0m" << std::endl;
00519     worker = std::thread(
00520         [this]{ loop(); }
00521     );
00522 }
00523
00524 void
00525 stop(){
00526     if( !is_exec.exchange(false) ){ return; }
00527
00528     if(
00529         worker.joinable()
00530     ){
00531         std::cout << "\033[1;32mSaindo da thread de leitura.\033[0m" << std::endl;
00532         worker.join();
00533     }
00534 }
00535 };
00536
00537 #endif // GPSTracker_HPP

```

6.9 src/main.cpp File Reference

Responsável por executar a aplicação.

```
#include "GPSTracker.hpp"
```

Include dependency graph for main.cpp:



Functions

- `int main (int argc, char *argv[])`

6.9.1 Detailed Description

Responsável por executar a aplicação.

Definition in file [main.cpp](#).

6.9.2 Function Documentation

6.9.2.1 main()

```
int main (  
    int argc,  
    char * argv[] )
```

Definition at line 7 of file [main.cpp](#).

6.10 main.cpp

[Go to the documentation of this file.](#)

```
00001  
00005 #include "GPSTracker.hpp"  
00006  
00007 int main(  
00008     int argc,  
00009     char* argv[]  
00010 ){  
00011  
00012     if(argc == 1){  
00013  
00014         std::cout << "Falta informar o IP e a PORTA de destino." << std::endl;  
00015         return -1;  
00016     }  
00017     else if(argc == 2){  
00018  
00019         std::cout << "Falta informar a PORTA de destino." << std::endl;  
00020         return -1;  
00021     }  
00022     else if(argc > 3){  
00023  
00024         std::cout << "Há argumentos inválidos, informe apenas IP e PORTA de destino." << std::endl;  
00025         return -1;  
00026     }  
00027  
00028     GPSTracker ss(  
00029         argv[1],  
00030         std::stoi(argv[2]),  
00031         "/dev/ttySTM2"  
00032     );  
00033  
00034     ss.init();  
00035  
00036     std::this_thread::sleep_for(std::chrono::seconds(60));  
00037  
00038     ss.stop();  
00039  
00040     return 0;  
00041 }
```


Index

- [__init__](#)
 - [GPSMonitor.GPSMonitor, 14](#)
- [~GPSSim](#)
 - [GPSSim, 19](#)
- [~GPSTracker](#)
 - [GPSTracker, 25](#)
- [addr_dest](#)
 - [GPSTracker, 28](#)
- [alt](#)
 - [GPSSim, 20](#)
- [altitude](#)
 - [GPSMonitor.GPSMonitor, 16](#)
- [build_nmea_string](#)
 - [GPSSim::NMEAGenerator, 31](#)
- [caminho_do_pseudo_terminal](#)
 - [GPSSim, 20](#)
- [conversion_utc_to_brasil](#)
 - [GPSMonitor.GPSMonitor, 14](#)
- [converter_lat_lon](#)
 - [GPSTracker::GPSData, 11](#)
- [data](#)
 - [GPSTracker::GPSData, 12](#)
- [debug.cpp](#)
 - [main, 35](#)
- [degrees_to_NMEA](#)
 - [GPSSim::NMEAGenerator, 31](#)
- [fd_filho](#)
 - [GPSSim, 21](#)
- [fd_pai](#)
 - [GPSSim, 21](#)
- [fd_serial](#)
 - [GPSTracker, 28](#)
- [format_integer](#)
 - [GPSSim::NMEAGenerator, 31](#)
- [generate_gga](#)
 - [GPSSim::NMEAGenerator, 32](#)
- [generate_rmc](#)
 - [GPSSim::NMEAGenerator, 32](#)
- [get_data_from_client](#)
 - [GPSMonitor.GPSMonitor, 15](#)
- [get_path_pseudo_term](#)
 - [GPSSim, 19](#)
- [get_utc_time](#)
 - [GPSSim::NMEAGenerator, 33](#)
- [gps_monitor](#)
 - [GPSMonitor, 7](#)
- [GPSData](#)
 - [GPSTracker::GPSData, 11](#)
- [GPSMonitor, 7](#)
 - [gps_monitor, 7](#)
- [GPSMonitor.GPSMonitor, 13](#)
 - [__init__, 14](#)
 - [altitude, 16](#)
 - [conversion_utc_to_brasil, 14](#)
 - [get_data_from_client, 15](#)
 - [img_data, 16](#)
 - [mainloop, 15](#)
 - [placeholder, 16](#)
 - [save_historic, 15](#)
 - [set_background, 15](#)
 - [sock, 16](#)
 - [time, 16](#)
 - [traject, 16](#)
 - [zona_brasileira, 16](#)
- [GPSSim, 17](#)
 - [~GPSSim, 19](#)
 - [alt, 20](#)
 - [caminho_do_pseudo_terminal, 20](#)
 - [fd_filho, 21](#)
 - [fd_pai, 21](#)
 - [get_path_pseudo_term, 19](#)
 - [GPSSim, 19](#)
 - [init, 19](#)
 - [is_exec, 21](#)
 - [lat, 21](#)
 - [lon, 21](#)
 - [loop, 20](#)
 - [stop, 20](#)
 - [to_move, 21](#)
 - [update, 20](#)
 - [worker, 21](#)
- [GPSSim::NMEAGenerator, 29](#)
 - [build_nmea_string, 31](#)
 - [degrees_to_NMEA, 31](#)
 - [format_integer, 31](#)
 - [generate_gga, 32](#)
 - [generate_rmc, 32](#)
 - [get_utc_time, 33](#)
- [GPSTracker, 22](#)
 - [~GPSTracker, 25](#)
 - [addr_dest, 28](#)
 - [fd_serial, 28](#)
 - [GPSTracker, 25](#)
 - [init, 26](#)

- ip_destino, 28
- is_exec, 28
- last_data_given, 28
- loop, 26
- open_serial, 26
- porta_destino, 28
- porta_serial, 29
- read_serial, 26
- send, 27
- sockfd, 29
- split, 27
- stop, 27
- worker, 29
- GPSTracker::GPSData, 9
 - converter_lat_lon, 11
 - data, 12
 - GPSData, 11
 - parsing, 11
 - pattern, 12
 - to_csv, 12
- img_data
 - GPSTracker.GPSTracker, 16
- init
 - GPSSim, 19
 - GPSTracker, 26
- ip_destino
 - GPSTracker, 28
- is_exec
 - GPSSim, 21
 - GPSTracker, 28
- last_data_given
 - GPSTracker, 28
- lat
 - GPSSim, 21
- lon
 - GPSSim, 21
- loop
 - GPSSim, 20
 - GPSTracker, 26
- main
 - debug.cpp, 35
 - main.cpp, 51
- main.cpp
 - main, 51
- mainloop
 - GPSTracker.GPSTracker, 15
- open_serial
 - GPSTracker, 26
- parsing
 - GPSTracker::GPSData, 11
- pattern
 - GPSTracker::GPSData, 12
- placeholder
 - GPSTracker.GPSTracker, 16
- porta_destino
 - GPSTracker, 28
- porta_serial
 - GPSTracker, 29
- read_serial
 - GPSTracker, 26
- save_historic
 - GPSTracker.GPSTracker, 15
- send
 - GPSTracker, 27
- set_background
 - GPSTracker.GPSTracker, 15
- sock
 - GPSTracker.GPSTracker, 16
- sockfd
 - GPSTracker, 29
- split
 - GPSTracker, 27
- src/debug.cpp, 35, 36
- src/GPSTracker.py, 36, 37
- src/GPSSim.hpp, 39, 41
- src/GPSTracker.hpp, 44, 46
- src/main.cpp, 50, 51
- stop
 - GPSSim, 20
 - GPSTracker, 27
- time
 - GPSTracker.GPSTracker, 16
- to_csv
 - GPSTracker::GPSData, 12
- to_move
 - GPSSim, 21
- traject
 - GPSTracker.GPSTracker, 16
- update
 - GPSSim, 20
- worker
 - GPSSim, 21
 - GPSTracker, 29
- zona_brasileira
 - GPSTracker.GPSTracker, 16